Generalización, selección de modelos y regularización

Tomas Santana
October, 2026

Índice

1. Parte 1: Selección de modelos 2
   1.1. Más features, más complejidad: regresión polinomial 2
   1.2. Por qué no podemos escoger con el error de entrenamiento 3
   1.3. Validación hold-out 3
      1.3.1. El conjunto de prueba 4
   1.4. Validación cruzada 4
   1.5. Baseline y data leakage 5
2. Parte 2: Regularización 6
   2.1. Idea general 6
   2.2. Regularización L_2: Ridge 6
      2.2.1. Centrar y escalar las features 6
      2.2.2. Solución cerrada 7
      2.2.3. El problema de la invertibilidad 8
      2.2.4. Descenso por gradiente: weight decay 8
      2.2.5. Ridge sobre el polinomio de grado 10 9
   2.3. Sparsity y regularización L_1: Lasso 9
      2.3.1. Modelos sparse 9
      2.3.2. ¿Por qué L_1 produce ceros y L_2 no? 10
      2.3.3. Caminos de coeficientes 11
   2.4. Escogiendo λ 12
   2.5. Ridge vs. Lasso 12
3. Parte 3: Interpretación probabilística 12
   3.1. Mínimos cuadrados como máxima verosimilitud 12
   3.2. Visión bayesiana: regularización como conocimiento previo (opcional) 14
      3.2.1. Prior gaussiano: Ridge 14
      3.2.2. Prior de Laplace: Lasso 15

Anteriormente se observó que, a medida que un modelo se vuelve más flexible, el error de entrenamiento siempre baja, pero el error de prueba primero baja y luego sube.

En este documento se responden tres preguntas que resultan de esta observación:

1. Si no podemos usar el error de entrenamiento para escoger la flexibilidad de un modelo, ¿qué usamos? (Parte 1)
2. ¿Hay alguna forma de controlar la flexibilidad de un modelo sin cambiar el número de parámetros? (Parte 2)
3. ¿Por qué usamos el error cuadrático y no otra función de costo, y qué relación tiene eso con la regularización? (Parte 3)

Seguimos con la notación anterior: tenemos m ejemplos de entrenamiento $\left(\mathbf{x}^{(i)}, y^{(i)}\right)$, el modelo tiene n parámetros $\theta_0, ..., \theta_{n-1}$, y la función de costo es

$$J(\theta) = \frac{1}{2}\sum_{i=1}^{m}\left(h_\theta\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right)^2.$$

Esta clase sigue de cerca las secciones 1.3 y 9.1 a 9.4 de las notas de Stanford CS229, y los capítulos 5 y 6 de ISLP.

1. Parte 1: Selección de modelos

1.1. Más features, más complejidad: regresión polinomial

Antes de hablar de cómo escoger un modelo, necesitamos una forma concreta de hacer que un modelo lineal sea más o menos flexible. La más sencilla es agregar features.

Supongamos que tenemos una sola variable de entrada x. En lugar de usar el modelo $h_\theta(x) = \theta_0 + \theta_1 x$, podemos usar un polinomio de grado d:

$$h_\theta(x) = \theta_0 + \theta_1 x + \theta_2 x^2 + \ldots + \theta_d x^d.$$

A primera vista esto no parece regresión lineal, porque h_θ no es una recta. Pero si definimos nuevas features

$$x_1 = x, \quad x_2 = x^2, \quad \ldots, \quad x_d = x^d,$$

el modelo queda $h_\theta(\mathbf{x}) = \theta^T \mathbf{x}$, exactamente igual que en la clase anterior, con $n = d + 1$ parámetros. La palabra "lineal" en regresión lineal se refiere a que el modelo es lineal en los parámetros θ, no en la variable x. Por eso todo lo que vimos (descenso por gradiente, ecuaciones normales) funciona sin cambios: solo cambia la matriz de diseño, que ahora tiene una columna por cada potencia:

$$\mathbf{X} = \begin{pmatrix} 1 & x^{(1)} & \left(x^{(1)}\right)^2 & \ldots & \left(x^{(1)}\right)^d \\ 1 & x^{(2)} & \left(x^{(2)}\right)^2 & \ldots & \left(x^{(2)}\right)^d \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x^{(m)} & \left(x^{(m)}\right)^2 & \ldots & \left(x^{(m)}\right)^d \end{pmatrix}$$

Un detalle práctico: si x está entre 0 y 5, entonces x^{10} puede llegar a casi diez millones, mientras que la primera columna vale 1. Con columnas de tamaños tan distintos, la matriz $\mathbf{X}^T\mathbf{X}$ es muy difícil de invertir numéricamente, y el descenso por gradiente converge muy lento. Por eso, antes de construir las potencias, es usual estandarizar x (restarle su media y dividir entre su desviación estándar). Volveremos a esto cuando veamos regularización.

La Figura 1 muestra qué pasa al ajustar polinomios de distintos grados a 15 ejemplos generados a partir de una función conocida más ruido.

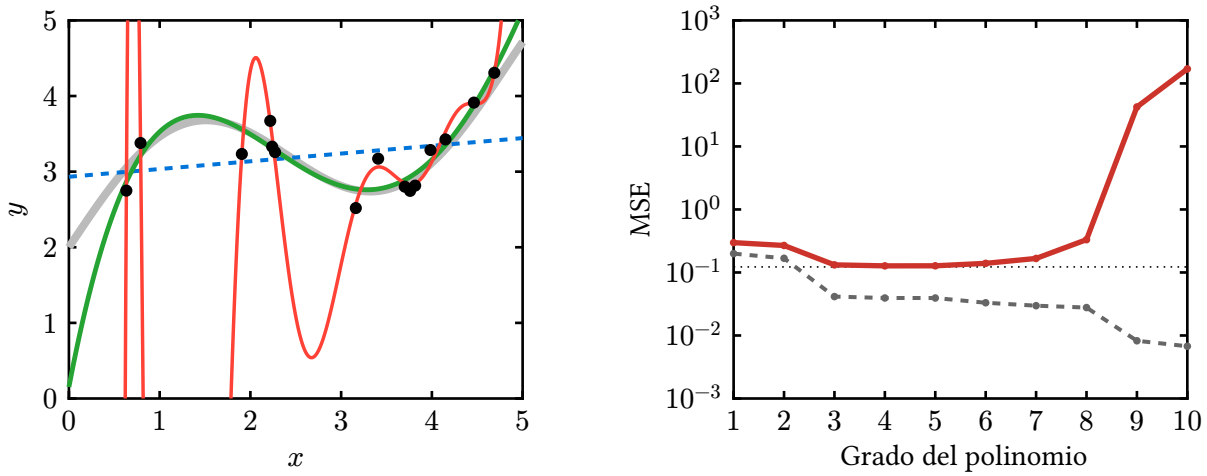


Figura 1: Regresión polinomial sobre 15 ejemplos de entrenamiento. A la izquierda, ajustes de grado 1 (azul), 4 (verde) y 10 (rojo); la curva gris es la función real. A la derecha, MSE de entrenamiento (gris punteado) y de validación (rojo) en función del grado, en escala logarítmica. La línea horizontal es la varianza del ruido, σ^2: ningún modelo puede tener un error esperado de validación menor.

La figura es la versión "real" de la Figura 1 de la clase anterior, ahora con ajustes calculados de verdad. Se observan tres cosas:

- El error de entrenamiento baja siempre que aumentamos el grado. Esto no es casualidad: un polinomio de grado $d+1$ puede imitar a cualquier polinomio de grado d (basta poner $\theta_{d+1} = 0$), así que su mejor ajuste nunca puede ser peor.
- El error de validación baja hasta grado 3 o 4, y luego sube. Con grado 10 el error de validación es más de mil veces mayor que con grado 4.
- El polinomio de grado 10 pasa muy cerca de los puntos de entrenamiento, pero entre ellos hace oscilaciones enormes. Sus coeficientes también son grandes: el mayor tiene magnitud 41.6, mientras que el mayor del polinomio de grado 4 tiene magnitud 2.8. Esta observación será el punto de partida de la Parte 2.

El grado d es un ejemplo de hiperparámetro: un valor que define al modelo pero que no se aprende con el algoritmo de entrenamiento, sino que lo escogemos nosotros. La pregunta de esta parte es cómo escogerlo.

1.2. Por qué no podemos escoger con el error de entrenamiento

Supongamos que tenemos varios modelos candidatos $M_1, \ldots, M_k$, por ejemplo, polinomios de grado $1, \ldots, 10$. La primera idea que se nos podría ocurrir es:

1. Entrenar cada modelo M_j sobre el conjunto de entrenamiento S.
2. Escoger el modelo con el menor error de entrenamiento.

Este procedimiento no funciona. Como acabamos de ver, el error de entrenamiento baja siempre que el modelo es más flexible, así que este método escogería siempre el polinomio de mayor grado, que es justamente el que peor generaliza.

El problema de fondo es que estamos usando los mismos datos para dos cosas: para ajustar los parámetros y para medir qué tan bueno es el ajuste. Necesitamos medir el error con datos que el modelo no haya usado para entrenarse.

1.3. Validación hold-out

La solución más sencilla es separar una parte de los datos y no usarla para entrenar. Este procedimiento se conoce como validación hold-out:

1. Dividir aleatoriamente S en dos partes: S_{train} (por ejemplo, el 70% de los datos) y S_{val} (el 30% restante). A S_{val} se le llama conjunto de validación.
2. Entrenar cada modelo M_j usando solamente S_{train}.
3. Escoger el modelo con el menor error sobre S_{val} (el error de validación).

Como los modelos no vieron los ejemplos de S_{val} durante el entrenamiento, el error de validación es una estimación razonable del error que tendrán con datos nuevos. Esto es exactamente lo que hicimos en la Figura 1: el grado 4 tiene el menor error de validación, así que es el que escogeríamos.

Usualmente se separa entre un cuarto y un tercio de los datos para validación. Cuando hay muchos datos, la fracción puede ser mucho menor, siempre que el número de ejemplos de validación sea suficiente. Por ejemplo, en ImageNet, con más de un millón de imágenes, es común usar unas 50 mil para validación, es decir, alrededor del 5%.

Una vez escogido el modelo, es buena idea volver a entrenarlo usando todos los datos (S_{train} y S_{val} juntos), ya que con más datos el ajuste suele ser mejor.

1.3.1. El conjunto de prueba

Hay un detalle sutil. Si probamos muchos modelos y escogemos el que tiene menor error de validación, ese error de validación tiende a ser optimista: escogimos ese modelo precisamente porque le fue bien en esos datos, en parte por suerte. Por eso, si queremos reportar qué tan bueno es el modelo final, necesitamos un tercer conjunto que no se use para nada durante el desarrollo: el conjunto de prueba.



Figura 2: División de los datos en entrenamiento, validación y prueba. El conjunto de entrenamiento se usa para ajustar los parámetros, el de validación para escoger hiperparámetros (como el grado del polinomio o λ), y el de prueba solamente para medir el desempeño del modelo final.

La regla es: el conjunto de prueba se usa una sola vez, al final. Si después de ver el error de prueba volvemos a cambiar el modelo, el conjunto de prueba se convirtió en otro conjunto de validación, y su error deja de ser una estimación honesta.

1.4. Validación cruzada

La desventaja de la validación hold-out es que desperdicia datos: los modelos se entrenan con el 70% de los datos, no con el 100%. Si tenemos muchos datos esto no importa, pero si tenemos pocos (por ejemplo, $m = 20$), perder el 30% puede cambiar mucho el resultado. Además, el error de validación depende de cuáles ejemplos quedaron en S_{val} por azar.

La validación cruzada k-fold resuelve esto usando todos los datos para validar, por turnos:

1. Dividir aleatoriamente S en k partes disjuntas $S_1, \ldots, S_k$, de m/k ejemplos cada una.
2. Para cada modelo M_j:
   - Para cada parte S_l, con $l = 1, \ldots, k$: entrenar M_j con todas las partes excepto S_l, y calcular su error sobre S_l.
   - Estimar el error de M_j como el promedio de esos k errores.
3. Escoger el modelo con el menor error estimado, y volver a entrenarlo con todo S.

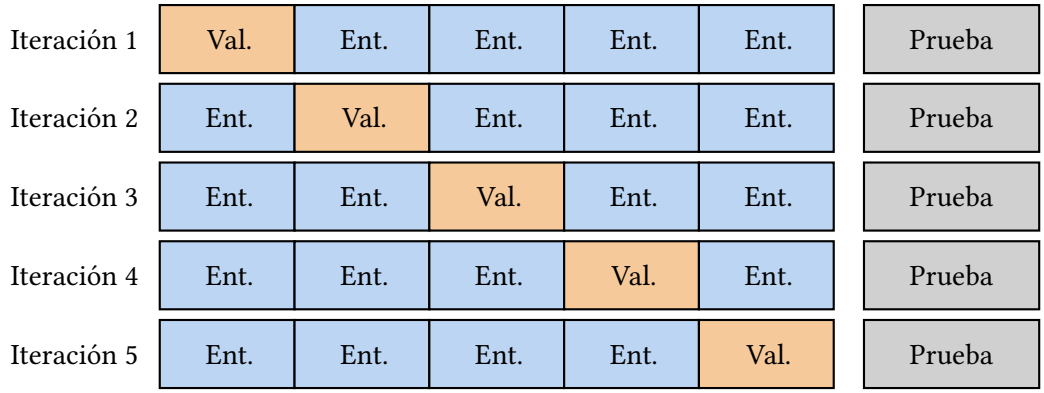


Figura 3: Validación cruzada con $k = 5$. En cada iteración, una parte distinta (naranja) se usa para validar y las demás (azul) para entrenar. El error estimado es el promedio de los 5 errores de validación. El conjunto de prueba (gris) queda fuera de todo el proceso.

Un valor típico es $k = 10$. Ahora en cada iteración solo se deja fuera $1/k$ de los datos, mucho menos que en hold-out. El costo es que cada modelo se entrena k veces.

Cuando hay muy pocos datos, se puede usar el caso extremo $k = m$: se entrena con todos los ejemplos menos uno, se mide el error en el ejemplo que quedó fuera, y se repite para cada ejemplo. Esto se conoce como leave-one-out cross validation (LOOCV).

La validación cruzada no solo sirve para escoger entre modelos. También se puede usar simplemente para estimar qué tan bueno es un único modelo cuando no tenemos suficientes datos para separar un conjunto de validación.

1.5. Baseline y data leakage

Antes de entrenar cualquier modelo, conviene tener un punto de comparación sencillo, llamado baseline. Para regresión, el baseline más simple es predecir siempre la media de las etiquetas de entrenamiento, $\hat{y} = \overline{y}$. Nótese que esto es exactamente regresión lineal con un solo parámetro, $h_\theta(\mathbf{x}) = \theta_0$: si minimizamos $J(\theta)$ respecto a θ_0, obtenemos $\theta_0 = \overline{y}$. Si un modelo más complicado no supera al baseline en validación, algo anda mal.

El otro error común en este proceso es el data leakage (fuga de datos): que información de los conjuntos de validación o prueba se filtre al entrenamiento. Cuando eso pasa, el error de validación o prueba sale mejor de lo que realmente es. Algunos ejemplos:

- Estandarizar las features usando la media y desviación estándar de todo el conjunto de datos, antes de separar entrenamiento y prueba. La media y la desviación estándar también se "aprenden" de los datos, así que deben calcularse solo con el conjunto de entrenamiento, y luego aplicarse tal cual a validación y prueba.
- Tener ejemplos duplicados que terminan uno en entrenamiento y otro en prueba.
- Usar features que no estarán disponibles al momento de predecir. Por ejemplo, para predecir el precio de una casa, usar la comisión que cobró el agente por venderla.
- Con datos que dependen del tiempo (como precios de acciones), separar los datos aleatoriamente. Si el modelo se entrena con datos de 2026 y se valida con datos de 2024, está usando el futuro para predecir el pasado. En estos casos se separa por fecha.

2. Parte 2: Regularización

2.1. Idea general

En la Parte 1, controlamos la complejidad del modelo cambiando el número de parámetros (el grado del polinomio). Pero la complejidad de un modelo no depende solo de cuántos parámetros tiene, sino también de qué tan grandes son. En la Figura 1, el polinomio de grado 10 tiene coeficientes grandes, y eso es lo que le permite oscilar tanto entre los puntos de entrenamiento.

La regularización consiste en agregar a la función de costo un término $R(\theta)$, llamado regularizador, que mide la complejidad del modelo:

$$J_\lambda(\theta) = J(\theta) + \lambda R(\theta).$$

El parámetro $\lambda \geq 0$ se llama parámetro de regularización, y controla el balance entre dos objetivos: ajustarse bien a los datos (que $J(\theta)$ sea pequeño) y tener un modelo sencillo (que $R(\theta)$ sea pequeño).

- Si $\lambda = 0$, recuperamos la función de costo original.
- Si λ es pequeño, el regularizador funciona casi como un criterio de desempate: entre modelos que se ajustan igual de bien a los datos, prefiere el más sencillo.
- Si λ es muy grande, el ajuste a los datos casi no importa, y el modelo termina siendo demasiado simple (alto sesgo).

Igual que el grado del polinomio, λ es un hiperparámetro, y se escoge con validación.

2.2. Regularización L_2: Ridge

El regularizador más usado es la suma de los cuadrados de los parámetros:

$$R(\theta) = \frac{1}{2}\sum_{j=1}^{n-1} \theta_j^2.$$

A la regresión lineal con este regularizador se le llama Ridge regression. Su función de costo es

$$J_\lambda(\theta) = \frac{1}{2}\sum_{i=1}^{m} \left(h_\theta\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right)^2 + \frac{\lambda}{2}\sum_{j=1}^{n-1} \theta_j^2.$$

Nótese que la suma empieza en $j = 1$: el intercepto θ_0 no se penaliza. La razón es que θ_0 no hace al modelo más flexible, solo lo sube o lo baja. Si lo penalizáramos, estaríamos empujando las predicciones hacia 0, y el resultado cambiaría según las unidades de y (no es lo mismo medir precios en dólares que en miles de dólares). Enseguida veremos que, preparando bien los datos, podemos sacar a θ_0 del problema.

2.2.1. Centrar y escalar las features

Antes de resolver Ridge, necesitamos preparar los datos. Hay dos razones.

La primera es que la penalización trata a todos los coeficientes por igual: θ_1^2 cuesta lo mismo que θ_2^2. Pero el tamaño de un coeficiente depende de las unidades de su feature. Si el área de una casa se mide en pies cuadrados en lugar de metros cuadrados, la feature se multiplica por 10.76 y su coeficiente se divide entre 10.76 para dar la misma predicción. Con un coeficiente más pequeño, la penalización sobre el área sería mucho menor, y el modelo regularizado cambiaría solo por haber cambiado de unidades.

Por eso, antes de regularizar, se estandariza cada feature:

$$x_j^{(i)} := \frac{x_j^{(i)} - \mu_j}{s_j}, \quad j = 1, ..., n-1,$$

donde μ_j y s_j son la media y la desviación estándar de la feature j. Después de esto, todas las features están en la misma escala y la penalización las trata de forma justa.

La segunda razón tiene que ver con θ_0. Después de estandarizar, cada feature tiene media cero en el conjunto de entrenamiento, es decir, $\sum_{i=1}^{m} x_j^{(i)} = 0$ para cada $j \geq 1$. Veamos qué pasa con la derivada de J_λ respecto a θ_0 (que no aparece en la penalización):

$$\frac{\partial J_\lambda(\theta)}{\partial \theta_0} = \sum_{i=1}^{m} \left(\theta_0 + \sum_{j=1}^{n-1} \theta_j x_j^{(i)} - y^{(i)} \right) = m\theta_0 + \sum_{j=1}^{n-1} \theta_j \underbrace{\sum_{i=1}^{m} x_j^{(i)}}_{=0} - \sum_{i=1}^{m} y^{(i)}.$$

Igualando a cero obtenemos

$$\theta_0 = \frac{1}{m} \sum_{i=1}^{m} y^{(i)} = \overline{y}.$$

Es decir, con las features centradas, el intercepto siempre es la media de las etiquetas, sin importar el valor de λ ni de los demás parámetros. Podemos fijarlo desde el principio y olvidarnos de él: basta con restarle $\overline{y}$ a todas las etiquetas y quitar la columna de unos de la matriz de diseño.

Así que, de aquí en adelante, en esta parte supondremos que:

- Las features están estandarizadas, y la matriz de diseño $\mathbf{X} \in \mathbb{R}^{m \times (n-1)}$ no tiene la columna de unos.
- Las etiquetas están centradas: $\mathbf{y}$ contiene los valores $y^{(i)} - \overline{y}$.
- $\theta = (\theta_1, ..., \theta_{n-1})^T$ contiene solo los parámetros que se penalizan.

Para predecir sobre un ejemplo nuevo $\mathbf{x}$, lo estandarizamos con las mismas μ_j y s_j, y calculamos $h_\theta(\mathbf{x}) = \overline{y} + \theta^T \mathbf{x}$.

Es importante que μ_j, s_j y $\overline{y}$ se calculen solo con el conjunto de entrenamiento, y luego se apliquen tal cual a los datos de validación y prueba. Si los calculáramos con todos los datos, tendríamos data leakage.

El escalamiento también ayuda al descenso por gradiente, aunque no haya regularización. Cuando las features tienen escalas muy distintas, las curvas de nivel de $J(\theta)$ son elipses muy alargadas, y el descenso por gradiente avanza en zigzag. Con features en la misma escala, las curvas de nivel son más parecidas a círculos y el algoritmo converge más rápido.

2.2.2. Solución cerrada

Con los datos centrados y escalados, la función de costo de Ridge en forma matricial es

$$J_\lambda(\theta) = \frac{1}{2}(\mathbf{X}\theta - \mathbf{y})^T(\mathbf{X}\theta - \mathbf{y}) + \frac{\lambda}{2}\theta^T\theta.$$

El gradiente del primer término ya lo calculamos en la clase anterior: $\mathbf{X}^T\mathbf{X}\theta - \mathbf{X}^T\mathbf{y}$. Para el segundo término, $\nabla_\theta \theta^T\theta = 2\theta$. Entonces:

$$\nabla_\theta J_\lambda(\theta) = \mathbf{X}^T\mathbf{X}\theta - \mathbf{X}^T\mathbf{y} + \lambda\theta.$$

Igualando a cero:

$$(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})\theta = \mathbf{X}^T\mathbf{y}$$

y por lo tanto

$$\theta = \left(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}\right)^{-1}\mathbf{X}^T\mathbf{y}.$$

Si $\lambda = 0$, recuperamos las ecuaciones normales de la clase anterior. La única diferencia con Ridge es que se le suma λ a la diagonal de $\mathbf{X}^T\mathbf{X}$.

2.2.3. El problema de la invertibilidad

En la clase anterior, al despejar $\theta = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}$, supusimos que $\mathbf{X}^T\mathbf{X}$ es invertible. Esto falla en dos situaciones comunes:

- Cuando hay menos ejemplos que parámetros. Por ejemplo, un polinomio de grado 20 con 15 ejemplos.
- Cuando alguna feature se puede escribir como combinación lineal de otras. Por ejemplo, si tenemos el área de la casa en metros cuadrados y también en pies cuadrados, una columna de $\mathbf{X}$ es la otra multiplicada por 10.76.

En ambos casos hay infinitas soluciones con el mismo error de entrenamiento, y las ecuaciones normales no nos dicen cuál escoger. Con Ridge esto no pasa: para cualquier $\lambda > 0$, la matriz $\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$ siempre es invertible. Veamos por qué.

Sea $\mathbf{M} = \mathbf{X}^T\mathbf{X} + \lambda\mathbf{I}$. Para cualquier vector $\mathbf{v} \neq 0$:

$$\mathbf{v}^T\mathbf{M}\mathbf{v} = \mathbf{v}^T\mathbf{X}^T\mathbf{X}\mathbf{v} + \lambda\mathbf{v}^T\mathbf{v} = \|\mathbf{X}\mathbf{v}\|^2 + \lambda\|\mathbf{v}\|^2 > 0,$$

porque el primer término es mayor o igual a cero y el segundo es estrictamente positivo. Esto implica que $\mathbf{M}$ es invertible: si no lo fuera, existiría un $\mathbf{v} \neq 0$ con $\mathbf{M}\mathbf{v} = 0$, y entonces $\mathbf{v}^T\mathbf{M}\mathbf{v} = 0$, lo cual es una contradicción.

Esta fue, de hecho, la motivación original de Ridge en estadística: arreglar el problema de invertibilidad. La interpretación como control de la complejidad vino después.

2.2.4. Descenso por gradiente: weight decay

También podemos minimizar J_λ con descenso por gradiente. La derivada respecto a θ_j es la misma de la clase anterior más la derivada del regularizador:

$$\frac{\partial J_\lambda(\theta)}{\partial \theta_j} = \sum_{i=1}^{m}\left(h_\theta\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right)x_j^{(i)} + \lambda\theta_j.$$

Sustituyendo en la regla de actualización:

$$\theta_j := \theta_j - \alpha\lambda\theta_j - \alpha\sum_{i=1}^{m}\left(h_\theta\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right)x_j^{(i)}$$

que podemos reescribir como

$$\theta_j := (1 - \alpha\lambda)\theta_j - \alpha\sum_{i=1}^{m}\left(h_\theta\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right)x_j^{(i)}.$$

Es decir, en cada paso primero multiplicamos cada parámetro por un número un poco menor que 1, y luego aplicamos la actualización LMS de siempre. Los parámetros se van "encogiendo" en cada paso, a menos que los datos los empujen a crecer. Por esta razón, en deep learning a la regularización L_2 se le llama weight decay (decaimiento de los pesos).

2.2.5. Ridge sobre el polinomio de grado 10

Volvamos al ejemplo de la Figura 1. Tomamos el polinomio de grado 10, que era el que peor generalizaba, y lo entrenamos con Ridge para distintos valores de λ.

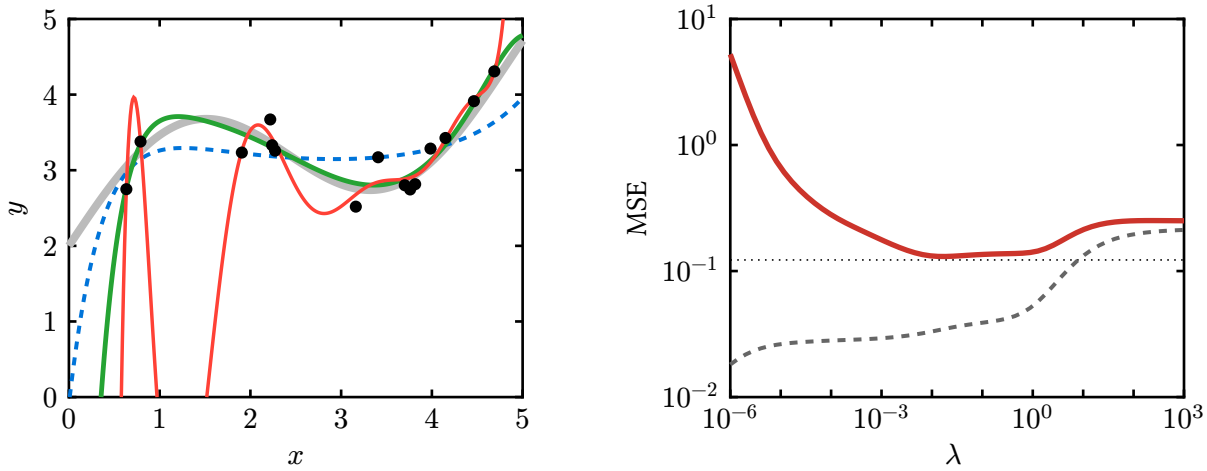


Figura 4: Ridge sobre el polinomio de grado 10 de la Figura 1. A la izquierda, ajustes con $\lambda = 10^{-6}$ (rojo), $\lambda \approx 0.019$ (verde, el de menor error de validación) y $\lambda = 10$ (azul). A la derecha, MSE de entrenamiento (gris punteado) y de validación (rojo) en función de λ. La complejidad del modelo disminuye de izquierda a derecha: la figura es la de la Figura 1, pero reflejada.

Con λ muy pequeño, el polinomio sigue oscilando entre los puntos de entrenamiento: sobreajusta. Con $\lambda \approx 0.019$, el polinomio de grado 10 tiene un error de validación de 0.131, casi igual al del polinomio de grado 4 sin regularizar. Con $\lambda = 10$ el modelo es demasiado rígido y subajusta.

Lo importante es que no tuvimos que cambiar el número de parámetros: el modelo sigue teniendo 11, pero λ controla qué tan libremente se pueden usar. El grado del polinomio y λ son dos perillas distintas para lo mismo: la complejidad del modelo.

2.3. Sparsity y regularización L_1: Lasso

2.3.1. Modelos sparse

La regularización también sirve para expresar algo que creemos sobre el problema. Supongamos que tenemos muchas features, pero creemos que solo unas pocas son realmente útiles para predecir y. En ese caso, nos gustaría un modelo donde la mayoría de los θ_j sean exactamente cero, de forma que el modelo solo use unas pocas features. A un modelo así se le llama sparse (disperso).

La forma directa de pedir esto sería penalizar el número de parámetros distintos de cero:

$$R(\theta) = \|\theta\|_0 = \text{número de } j \text{ tales que } \theta_j \neq 0.$$

El problema es que esta función no es continua: salta de 0 a 1 en cuanto un parámetro deja de ser cero, sin importar si se volvió 0.001 o 1000. No podemos minimizarla con descenso por gradiente. Por eso, en la práctica se reemplaza por la suma de los valores absolutos:

$$R(\theta) = \|\theta\|_1 = \sum_{j=1}^{n-1} |\theta_j|.$$

A la regresión lineal con este regularizador se le llama Lasso. Igual que con Ridge, las features se centran y escalan antes de ajustar el modelo, y el intercepto es $\overline{y}$:

$$J_\lambda(\theta) = \frac{1}{2}\sum_{i=1}^{m}\left(h_\theta\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right)^2 + \lambda\sum_{j=1}^{n-1}|\theta_j|.$$

2.3.2. ¿Por qué L_1 produce ceros y L_2 no?

Una primera intuición: si nos restringimos a los puntos (θ_1, θ_2) que están a distancia 1 del origen (el círculo unitario), los puntos con menor norma L_1 son $(\pm 1, 0)$ y $(0, \pm 1)$, que son justamente los que tienen un solo coeficiente distinto de cero. Es decir, minimizar la norma L_1 y buscar modelos sparse apuntan en la misma dirección.

Para ver el efecto sobre la regresión, es útil escribir los dos problemas de otra forma. En lugar de sumar la penalización a la función de costo, podemos poner un límite $t > 0$ al tamaño de los parámetros y minimizar $J(\theta)$ solo entre los θ que lo respetan. Para Ridge:

$$\min_\theta J(\theta) \quad \text{sujeto a} \quad \sum_{j=1}^{n-1}\theta_j^2 \leq t,$$

y para Lasso:

$$\min_\theta J(\theta) \quad \text{sujeto a} \quad \sum_{j=1}^{n-1}|\theta_j| \leq t.$$

Estas formulaciones son equivalentes a las anteriores: para cada λ hay un t que da la misma solución. La relación va en sentido contrario: un λ grande corresponde a un t pequeño, es decir, a una región permitida pequeña. Con $\lambda = 0$ no hay restricción, lo que corresponde a un t tan grande que el mínimo sin regularizar ya la cumple.

Con dos parámetros, θ_1 y θ_2, podemos dibujar las regiones permitidas:

- Para Ridge, $\theta_1^2 + \theta_2^2 \leq t$ es un círculo.
- Para Lasso, $|\theta_1| + |\theta_2| \leq t$ es un rombo.

La solución es el punto de la región permitida que toca la curva de nivel más baja de $J(\theta)$. En la Figura 5 la función $J(\theta)$ es la misma en ambos paneles. El círculo no tiene esquinas, así que la curva de nivel lo toca en un punto cualquiera, donde ambos parámetros son distintos de cero. El rombo tiene esquinas sobre los ejes, y es muy común que la curva de nivel lo toque justo en una esquina, donde uno de los parámetros es exactamente cero.

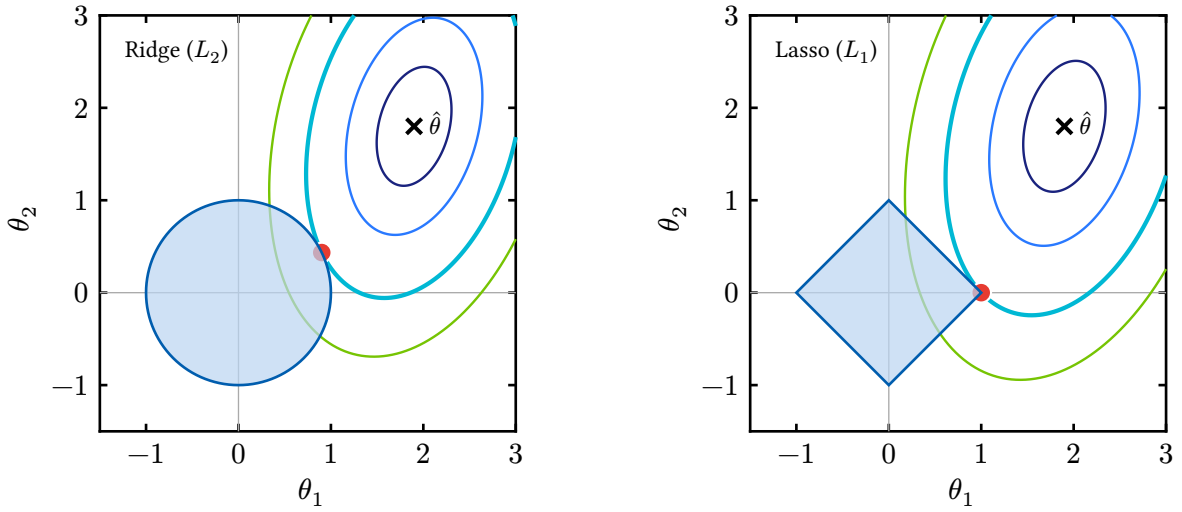


Figura 5: Curvas de nivel de una misma función $J(\theta)$ y región permitida por el regularizador. La $\times$ marca el mínimo sin regularizar, $\hat{\theta}$, y el punto rojo la solución regularizada: el punto de la región permitida donde J es menor. La curva de nivel más gruesa es la que toca la región permitida. Con Ridge (izquierda) la solución tiene ambos parámetros distintos de cero. Con Lasso (derecha) la solución cae en una esquina del rombo, donde $\theta_2 = 0$.

2.3.3. Caminos de coeficientes

Otra forma de ver la diferencia es graficar cómo cambia cada coeficiente a medida que aumentamos λ. A estas curvas se les llama caminos de regularización. Para la Figura 6 generamos 80 casas con ocho features: área, habitaciones, baños, antigüedad, distancia al centro, y tres features de ruido que no tienen ninguna relación con el precio. El área y el número de habitaciones están correlacionadas, como en la realidad.

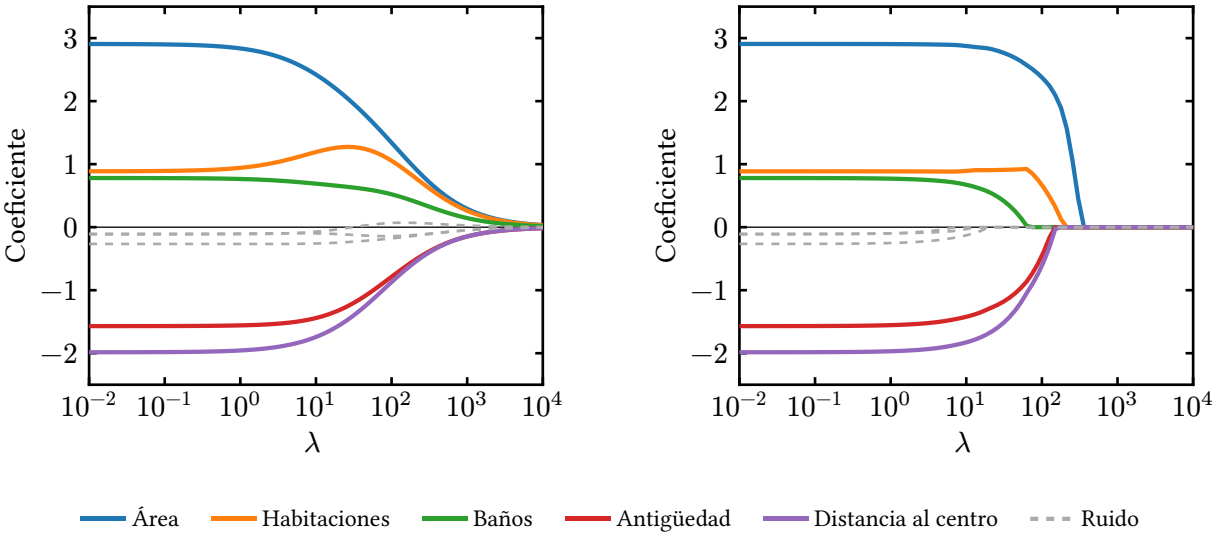


Figura 6: Caminos de regularización de Ridge (izquierda) y Lasso (derecha) para el problema de las casas, con las features estandarizadas. Las líneas grises punteadas son las tres features de ruido. Con Ridge, todos los coeficientes se encogen poco a poco y ninguno llega a ser exactamente cero. Con Lasso, los coeficientes van llegando a cero uno por uno, y los primeros en desaparecer son los de las features de ruido.

Con Lasso, las tres features de ruido son las primeras en desaparecer: sus coeficientes son exactamente cero para todo λ mayor que 22. En cambio, el área (la feature más importante) es la última, y su coeficiente sobrevive hasta $\lambda \approx 361$. Por eso se dice que Lasso hace selección de features

de forma automática. Ridge, en cambio, mantiene todas las features en el modelo, con coeficientes cada vez más pequeños. Algo curioso pasa con las habitaciones en Ridge: su coeficiente crece un poco antes de encogerse. Como está correlacionada con el área, cuando la penalización encoge el coeficiente del área, parte de ese peso se pasa a las habitaciones.

Lasso tiene una desventaja: la función $|\theta_j|$ no es derivable en $\theta_j = 0$, así que no tiene solución cerrada ni podemos aplicar directamente el descenso por gradiente que vimos. Se usan otros algoritmos (por ejemplo, scikit-learn usa descenso por coordenadas, que optimiza un parámetro a la vez), pero no entraremos en esos detalles.

2.4. Escogiendo λ

El valor de λ se escoge con validación, igual que el grado del polinomio en la Parte 1:

1. Escoger una lista de valores candidatos. Como no sabemos ni el orden de magnitud de λ, se usa una escala logarítmica, por ejemplo $10^{-4}, 10^{-3}, \ldots, 10^{3}$.
2. Para cada valor, estimar el error con validación hold-out o validación cruzada.
3. Escoger el λ con menor error y volver a entrenar con todos los datos de entrenamiento.

Esto es exactamente lo que muestra el panel derecho de la Figura 4.

2.5. Ridge vs. Lasso

	Ridge	**Lasso**
Regularizador	$\frac{1}{2}\sum_{j\geq 1}\theta_j^2$	$\sum_{j\geq 1}\lvert\theta_j\rvert$
Solución cerrada	Sí	No
Coeficientes exactamente cero	No	Sí
Features correlacionadas	Reparte el peso	Tiende a escoger una

En la práctica, Ridge suele funcionar mejor cuando muchas features aportan un poco cada una, y Lasso cuando creemos que solo unas pocas importan, o cuando queremos un modelo fácil de interpretar. También existe una combinación de ambos, llamada Elastic Net, que suma los dos regularizadores.

Una advertencia para el laboratorio: scikit-learn no usa exactamente la misma escala de λ que estas notas. Con nuestra definición de $J(\theta)$, el parámetro `alpha` de `Ridge` coincide con λ, pero el de `Lasso` corresponde a λ/m, porque `Lasso` divide el error entre el número de ejemplos. Por eso, un mismo valor de `alpha` no significa lo mismo en ambas clases.

3. Parte 3: Interpretación probabilística

En la clase anterior justificamos el error cuadrático con dos argumentos intuitivos: los errores positivos y negativos no se cancelan, y los errores grandes se penalizan más. Pero hay otras funciones con esas propiedades (por ejemplo, el valor absoluto o la cuarta potencia). En esta parte veremos que, bajo ciertas suposiciones sobre cómo se generan los datos, el error cuadrático sale de forma natural. Esta idea será la base para construir la función de costo de clasificación en la próxima clase.

3.1. Mínimos cuadrados como máxima verosimilitud

Supongamos que las etiquetas se generan así:

$$y^{(i)} = \theta^T \mathbf{x}^{(i)} + \varepsilon^{(i)},$$

donde $\varepsilon^{(i)}$ es un término de error. Este error representa todo lo que el modelo no captura: features que no incluimos (por ejemplo, si la casa tiene vista al lago) o simplemente ruido aleatorio. Supongamos además que los $\varepsilon^{(i)}$ son independientes entre sí y siguen todos la misma distribución normal con media cero y varianza σ^2, lo que escribimos $\varepsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$. La densidad de cada error es

$$p\big(\varepsilon^{(i)}\big) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{\big(\varepsilon^{(i)}\big)^2}{2\sigma^2}\right).$$

Como $\varepsilon^{(i)} = y^{(i)} - \theta^T \mathbf{x}^{(i)}$, esto significa que, dado $\mathbf{x}^{(i)}$, la etiqueta $y^{(i)}$ sigue una distribución normal centrada en la predicción del modelo:

$$p\big(y^{(i)} \mid \mathbf{x}^{(i)}; \theta\big) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{\big(y^{(i)} - \theta^T \mathbf{x}^{(i)}\big)^2}{2\sigma^2}\right).$$

Puede notarse también que para la distribución de $y^{(i)}$, la media es $\theta^T \mathbf{x}^{(i)}$ y la varianza es σ^2, que no depende de $\mathbf{x}^{(i)}$.

El punto y coma en $p\big(y^{(i)} \mid \mathbf{x}^{(i)}; \theta\big)$ indica que θ no es una variable aleatoria, sino un valor fijo (aunque desconocido) que parametriza la distribución. La Figura 7 ilustra este modelo.

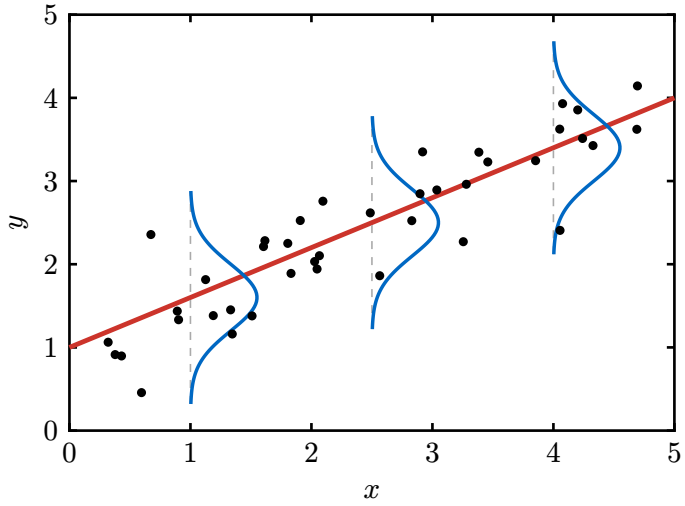


Figura 7: Modelo probabilístico de la regresión lineal. Para cada x, la etiqueta y sigue una distribución normal centrada en la recta $\theta^T \mathbf{x}$ (las campanas, dibujadas de lado) con la misma varianza σ^2 en todos los puntos.

Dados todos los datos, ¿cuál es la probabilidad de haber observado justamente las etiquetas $\mathbf{y}$? Vista como función de θ, esta probabilidad se llama verosimilitud (likelihood). Como los errores son independientes, es el producto de las densidades de cada ejemplo:

$$L(\theta) = p(\mathbf{y} \mid \mathbf{X}; \theta) = \prod_{i=1}^{m} p\big(y^{(i)} \mid \mathbf{x}^{(i)}; \theta\big) = \prod_{i=1}^{m} \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{\big(y^{(i)} - \theta^T \mathbf{x}^{(i)}\big)^2}{2\sigma^2}\right).$$

El principio de máxima verosimilitud dice que debemos escoger el θ que hace más probables los datos que observamos, es decir, el que maximiza $L(\theta)$.

Maximizar $L(\theta)$ es lo mismo que maximizar cualquier función creciente de $L(\theta)$. Es más cómodo maximizar su logaritmo, porque convierte el producto en una suma:

$$\ell(\theta) = \log L(\theta) = \sum_{i=1}^{m} \log\left(\frac{1}{\sqrt{2\pi}\sigma}\exp\left(-\frac{\left(y^{(i)} - \theta^T \mathbf{x}^{(i)}\right)^2}{2\sigma^2}\right)\right)$$
$$= m\log\frac{1}{\sqrt{2\pi}\sigma} - \frac{1}{\sigma^2}\cdot\frac{1}{2}\sum_{i=1}^{m}\left(y^{(i)} - \theta^T \mathbf{x}^{(i)}\right)^2.$$

El primer término no depende de θ, y $1/\sigma^2$ es una constante positiva. Por lo tanto, maximizar $\ell(\theta)$ es exactamente lo mismo que minimizar

$$\frac{1}{2}\sum_{i=1}^{m}\left(y^{(i)} - \theta^T \mathbf{x}^{(i)}\right)^2 = J(\theta).$$

Es decir, si suponemos que los errores son gaussianos e independientes, minimizar el error cuadrático es lo mismo que encontrar el estimador de máxima verosimilitud de θ. Nótese también que el resultado no depende de σ^2: obtendríamos el mismo θ aunque no conociéramos la varianza del ruido.

Esto no significa que el error cuadrático solo tenga sentido cuando el ruido es gaussiano. Es una justificación, no un requisito. Lo valioso es la receta: escoger una distribución para y, escribir la verosimilitud, y maximizarla. Si en lugar de una normal suponemos otra distribución, obtenemos otra función de costo.

3.2. Visión bayesiana: regularización como conocimiento previo (opcional)

En la sección anterior tratamos θ como un valor fijo pero desconocido. Esta es la visión frecuentista. Otra forma de ver el problema es la visión bayesiana: tratar θ como una variable aleatoria, y expresar lo que creemos sobre sus valores antes de ver los datos mediante una distribución $p(\theta)$, llamada distribución a priori (prior).

Después de observar los datos S, actualizamos esa creencia con el teorema de Bayes y obtenemos la distribución a posteriori:

$$p(\theta \mid S) = \frac{p(S \mid \theta)p(\theta)}{p(S)} = \frac{\left(\prod_{i=1}^{m} p\left(y^{(i)} \mid \mathbf{x}^{(i)}, \theta\right)\right)p(\theta)}{p(S)}.$$

Calcular la distribución a posteriori completa suele ser muy difícil, porque $p(S)$ requiere integrar sobre todos los valores posibles de θ. Una aproximación común es quedarse solo con el valor de θ más probable según la posteriori. A este valor se le llama estimador MAP (máximo a posteriori):

$$\theta_{\text{MAP}} = \arg\max_{\theta} \prod_{i=1}^{m} p\left(y^{(i)} \mid \mathbf{x}^{(i)}, \theta\right)p(\theta).$$

Como $p(S)$ no depende de θ, no hace falta calcularlo. Nótese también que ahora escribimos $p\left(y^{(i)} \mid \mathbf{x}^{(i)}, \theta\right)$ con coma y no con punto y coma: como θ ahora es una variable aleatoria, sí tiene sentido condicionar en su valor. Nótese que es la misma fórmula que la de máxima verosimilitud, con un factor adicional: el prior $p(\theta)$.

3.2.1. Prior gaussiano: Ridge

Supongamos que, antes de ver los datos, creemos que los parámetros (excepto θ_0) probablemente son pequeños. Podemos expresar esto con un prior normal centrado en cero, con varianza τ^2:

$$\theta_j \sim \mathcal{N}(0, \tau^2), \quad j = 1, \ldots, n-1.$$

Tomando el logaritmo del objetivo MAP:

$$\log\left(\prod_{i=1}^{m} p\big(y^{(i)} \mid \mathbf{x}^{(i)}, \theta\big)p(\theta)\right) = \ell(\theta) + \sum_{j=1}^{n-1} \log p\big(\theta_j\big) = \text{constante} - \frac{1}{\sigma^2} J(\theta) - \frac{1}{2\tau^2}\sum_{j=1}^{n-1}\theta_j^2.$$

Maximizar esto es lo mismo que minimizar lo que está restando. Multiplicando por σ^2:

$$\theta_{\text{MAP}} = \arg\min_{\theta} J(\theta) + \frac{\sigma^2}{\tau^2} \cdot \frac{1}{2}\sum_{j=1}^{n-1}\theta_j^2.$$

Esto es exactamente Ridge, con $\lambda = \sigma^2/\tau^2$.

- Si τ es pequeño, creemos con mucha seguridad que los parámetros son cercanos a cero, y eso corresponde a un λ grande.
- Si τ es grande, nuestro prior es casi plano (no creemos casi nada de antemano), y λ se acerca a cero, es decir, volvemos a máxima verosimilitud.
- Si el ruido σ^2 es grande, los datos son poco confiables, y el prior pesa más.

3.2.2. Prior de Laplace: Lasso

Si en lugar de una normal usamos una distribución de Laplace, $p\big(\theta_j\big) \propto \exp\big(-|\theta_j|/b\big)$, que tiene un pico más agudo en cero, el mismo procedimiento da

$$\theta_{\text{MAP}} = \arg\min_{\theta} J(\theta) + \frac{\sigma^2}{b}\sum_{j=1}^{n-1}|\theta_j|,$$

que es Lasso con $\lambda = \sigma^2/b$. El pico agudo en cero del prior de Laplace es la versión probabilística de las esquinas del rombo en la Figura 5.